

De acuerdo. Para el **Reporte 1 — EDA / Data Readiness** vamos a trabajar la población analítica como **un único dataset**, sin mencionar todavía ningún esquema de partición temporal para modelado. Tampoco usaremos “victim/non-victim”; hablaremos de **positive/event** y **negative/non-event**, porque el outcome es la primera alerta calificante observada.

La fuente principal para el EDA puede ser:

```
bu_frd.model_features_ab_final_2022_2025
```

porque ya representa la población final a nivel **party_id**, y para controles de integridad del outcome utilizaremos también:

```
bu_frd.model_cohort_behavioral_final_3m_2022_2025  
bu_frd.party_first_valid_alert_all
```

Empecemos por EDA 1 — Dataset Overview & Class Balance

Primero necesitamos congelar las métricas básicas del universo completo. Ejecuta esto en R/Spark:

```
eda_class_balance <- sdf_sql(  
  sc,  
  "  
  SELECT  
    COUNT(*) AS total_records,  
    COUNT(DISTINCT party_id) AS unique_parties,  
  
    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,  
    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,  
  
    AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate,  
    AVG(CASE WHEN label_3m = 0 THEN 1.0 ELSE 0.0 END) AS negative_rate,  
  
    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) * 1.0 /  
    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS negative_to_positive_ratio  
  
  FROM bu_frd.model_features_ab_final_2022_2025  
  "  
) %>%  
  collect()  
  
eda_class_balance
```

De acuerdo con los controles que ya tenemos, deberíamos quedar alrededor de:

Metric	Expectad
Total records	688,589
Unique parties	688,589
Positive cases	62,599
Negative cases	625,990
Positive rate	9.09%
Negative rate	90.91%
Negative : Positive	10 : 1

Pero para el reporte final utilizaremos **los resultados que arroje el query**, no números escritos manualmente.

Hay una consideración importante para la narrativa del EDA: ese **9.09%** es la composición de nuestra **muestra analítica**. No debemos presentarlo como prevalencia poblacional de fraud alerts.

EDA 2 — Basic Dataset Integrity

Antes de cualquier gráfico de comportamiento, confirmemos que la unidad analítica está limpia:

```
eda_basic_integrity <- sdf_sql(
  sc,
  "
  SELECT

    COUNT(*) AS total_rows,

    COUNT(DISTINCT party_id) AS distinct_parties,

    COUNT(*) - COUNT(DISTINCT party_id) AS duplicate_party_rows,

    SUM(
      CASE
        WHEN party_id IS NULL THEN 1
        ELSE 0
      END
    ) AS null_party_id,
```

```

SUM(
  CASE
    WHEN observation_date IS NULL THEN 1
    ELSE 0
  END
) AS null_observation_date,

SUM(
  CASE
    WHEN label_3m IS NULL THEN 1
    ELSE 0
  END
) AS null_label,

SUM(
  CASE
    WHEN label_3m NOT IN (0,1) THEN 1
    ELSE 0
  END
) AS invalid_label

FROM bu_frd.model_features_ab_final_2022_2025
"
) %>%
collect()

```

eda_basic_integrity

Idealmente queremos:

```

duplicate_party_rows = 0
null_party_id        = 0
null_observation_date = 0
null_label           = 0
invalid_label        = 0

```

Esto se convertirá posteriormente en una pequeña sección de **Data Integrity KPI cards**, no necesariamente en una gráfica.

EDA 3 — Outcome Integrity

Este QA es más importante que el simple null check. Queremos demostrar que el label realmente respeta la definición temporal:

```

\[ label=1 \iff observation\_date < first\_alert\_date \leq
LAST\_DAY(ADD\_MONTHS(observation\_date,3)) \]

```

Ejecuta:

```
eda_label_integrity <- sdf_sql(  
  sc,  
  "  
  WITH base AS (  
  
    SELECT  
      party_id,  
      observation_date,  
      label_3m  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
  ),  
  
  alerts AS (  
  
    SELECT  
      party_id,  
      first_alert_date  
  
    FROM bu_frd.party_first_valid_alert_all  
  
  )  
  
  SELECT  
  
    COUNT(*) AS total_rows,  
  
    SUM(  
      CASE  
        WHEN b.label_3m = 1  
          AND a.first_alert_date IS NULL  
        THEN 1 ELSE 0  
      END  
    ) AS positive_without_alert,  
  
    SUM(  
      CASE  
        WHEN b.label_3m = 1  
          AND a.first_alert_date <= b.observation_date  
        THEN 1 ELSE 0  
      END  
    ) AS positive_alert_not_future,  
  
    SUM(  
      CASE
```

```

        WHEN b.label_3m = 1
        AND a.first_alert_date >
            LAST_DAY(ADD_MONTHS(b.observation_date,3))
        THEN 1 ELSE 0
    END
) AS positive_alert_outside_horizon,

SUM(
    CASE
        WHEN b.label_3m = 0
        AND a.first_alert_date > b.observation_date
        AND a.first_alert_date <=
            LAST_DAY(ADD_MONTHS(b.observation_date,3))
        THEN 1 ELSE 0
    END
) AS negative_with_alert_in_horizon

FROM base b

LEFT JOIN alerts a
    ON b.party_id = a.party_id
"
) %>%
collect()

eda_label_integrity

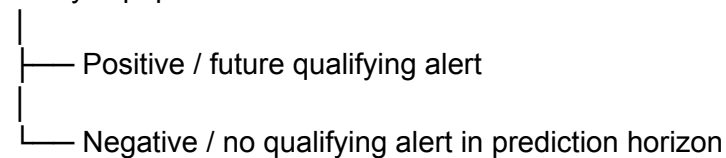
```

Aquí idealmente todos estos errores deben ser 0.

Primera gráfica del EDA

Con esta primera sección podemos construir después una gráfica muy sencilla:

Analytic population



Para la presentación probablemente usaría **dos KPI cards + una barra de composición**, no un pie chart, porque la diferencia 10:1 se compara mejor visualmente con barras.

También podemos incorporar un KPI:

Unique party coverage = 100%

si confirmamos:

$$TotalRecords = UniqueParties$$

2-Siguiente sección: Temporal Cohort Profile

Sin hablar todavía de particiones de modelado, debemos revisar cómo está distribuida la población por `observation_date`. Esto nos permite detectar concentraciones, huecos o cambios fuertes a través del tiempo.

Primero, resumen anual:

```
eda_cohort_year <- sdf_sql(
  sc,
  "
  SELECT
    YEAR(observation_date) AS observation_year,

    COUNT(*) AS total_parties,

    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,

    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,

    AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate

  FROM bu_frd.model_features_ab_final_2022_2025

  GROUP BY YEAR(observation_date)

  ORDER BY observation_year
  "
) %>%
  collect()

eda_cohort_year
```

Y después a nivel mensual:

```
eda_cohort_month <- sdf_sql(
  sc,
  "
  SELECT
    YEAR(observation_date) AS observation_year,
    MONTH(observation_date) AS observation_month,

    COUNT(*) AS total_parties,

    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,
```

```

SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,

AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate

FROM bu_frd.model_features_ab_final_2022_2025

GROUP BY
  YEAR(observation_date),
  MONTH(observation_date)

ORDER BY
  observation_year,
  observation_month
"
) %>%
collect()

```

eda_cohort_month

Y un pequeño control del rango:

```

eda_date_range <- sdf_sql(
  sc,
  "
SELECT
  MIN(observation_date) AS min_observation_date,
  MAX(observation_date) AS max_observation_date,
  COUNT(DISTINCT observation_date) AS distinct_observation_dates

FROM bu_frd.model_features_ab_final_2022_2025
"
) %>%
collect()

```

eda_date_range

Con estos resultados podremos construir dos visuales posteriores:

Population volume over calendar time
Positive rate over calendar time

La segunda será particularmente importante: no queremos asumir que la tasa analítica es constante a través del tiempo.

Después de esta sección avanzaría a **Tenure / observation-history confounding**, pero primero quiero ver estos resultados temporales para no mezclar posibles efectos de calendario con tenure.

3-Siguiente sección: Tenure / Observation-History Confounding

Aquí sí tenemos que ser cuidadosos porque previamente identificamos que la fuente utilizada para tenure está expuesta a **left censoring**. Por eso no quiero construir todavía buckets arbitrarios sin primero conocer la distribución de la variable disponible.

El siguiente paso debería ser primero identificar qué información de tenure quedó disponible en nuestra cohorte final. Ejecutemos:

```
eda_tenure_profile <- sdf_sql(  
  sc,  
  "  
  SELECT  
    COUNT(*) AS total_parties,  
  
    SUM(CASE WHEN tenure_months IS NULL THEN 1 ELSE 0 END)  
    AS tenure_null_count,  
  
    AVG(tenure_months) AS avg_tenure_months,  
  
    MIN(tenure_months) AS min_tenure_months,  
  
    percentile_approx(tenure_months, 0.01) AS p01,  
    percentile_approx(tenure_months, 0.05) AS p05,  
    percentile_approx(tenure_months, 0.25) AS p25,  
    percentile_approx(tenure_months, 0.50) AS p50,  
    percentile_approx(tenure_months, 0.75) AS p75,  
    percentile_approx(tenure_months, 0.95) AS p95,  
    percentile_approx(tenure_months, 0.99) AS p99,  
  
    MAX(tenure_months) AS max_tenure_months  
  
  FROM bu_frd.model_features_ab_final_2022_2025  
  "  
) %>%  
  collect()
```

eda_tenure_profile

Pero antes de ejecutarlo, verifica:

```
colnames(  
  tbl(sc, "bu_frd.model_features_ab_final_2022_2025")  
)
```

porque no quiero asumir que `tenure_months` terminó físicamente dentro de la tabla final. Si no aparece, **no ejecutes el primer query**. Pásame los nombres relacionados con `tenure/person history` que aparezcan, o simplemente dime que no existe.

Esto es intencional: para el EDA prefiero volver a la fuente correcta de `tenure` si es necesario, en lugar de reconstruir una métrica con una columna equivocada.

Estado del EDA hasta ahora

Tenemos ya dos bloques que pueden considerarse cerrados:

Dataset Structure & Outcome Integrity — PASS

- 688,589 observaciones.
- 688,589 parties únicos.
- 0 duplicados.
- 0 labels inválidos.
- 0 violaciones del horizonte temporal del outcome.

Temporal Coverage — PASS, with cohort-volume concentration noted

- 48 meses consecutivos.
- Enero 2022 → diciembre 2025.
- Sin gaps mensuales.
- Variación importante del volumen analítico.
- Concentración particularmente alta en mayo–junio 2024.
- Positive rate constante por diseño de sampling, por lo que **no se utilizará como evidencia de estabilidad temporal**.

Este último punto es una mejora importante respecto al EDA framework original.

3.1.

Perfecto. Eso confirma que `tenure_months` **no forma parte del dataset final de features**, y está bien: para este EDA podemos tratar `tenure` como una **variable auxiliar de diagnóstico**, no como predictor del modelo.

Además, esto evita modificar `bu_frd.model_features_ab_final_2022_2025` solo para hacer el reporte.

El siguiente paso es volver a la fuente donde evaluamos `tenure`, `bc3.person_restricted`, pero antes necesito que confirmemos exactamente qué columnas de fechas están disponibles actualmente. Ejecuta únicamente este diagnóstico:

```
person_restricted_cols <- colnames(  
  
tbl(sc, "bc3.person_restricted")
```

)

person_restricted_cols

Si la lista es muy larga, puedes usar:

person_restricted_cols[

grepl(

"date|dte|create|start|open|first|party",

person_restricted_cols,

ignore.case = TRUE

)

]

También puedes obtener schema y tipos:

person_restricted_schema <- sdf_sql(

sc,

"

DESCRIBE bc3.person_restricted

"

) %>%

collect()

person_restricted_schema

No construiría todavía **tenure_months** hasta ver cuál es exactamente la fecha disponible.

Por qué lo hacemos separado

Nuestro EDA quedaría conceptualmente:

FINAL ANALYTIC COHORT

bu_frd.model_features_ab_final_2022_2025

|
| party_id



TENURE AUXILIARY SOURCE

bc3.person_restricted

Luego calcularemos algo conceptualmente parecido a:

$$TenureMonths = months_between(observation_date, first_observed_customer_date)$$

pero **solo después de verificar qué representa realmente esa fecha.**

Esto es importante porque anteriormente encontramos que esta fuente tiene un problema potencial de **left censoring**. Es decir, para algunos clientes la primera fecha observada puede representar simplemente el inicio de nuestra disponibilidad histórica y no necesariamente el verdadero inicio de su relación.

Por eso en el reporte no deberíamos decir:

“Customer tenure is X months”

sin más.

Sería más correcto llamarlo algo como:

Observed tenure / available customer history

y separar, si es posible:

reliably observed tenure

vs

left-censored tenure

No necesitamos alterar el resto del EDA

Hasta ahora seguimos teniendo cerrados:

- | | |
|-------------------------------------|------|
| 1. Dataset Overview & Class Balance | PASS |
| 2. Dataset Integrity | PASS |
| 3. Outcome Integrity | PASS |

4. Temporal Coverage PASS

Estamos únicamente resolviendo:

5. Tenure / Observed-History Confounding

Una vez me pases las columnas relacionadas con fechas de `bc3.person_restricted`, te doy el query exacto para construir los buckets y comparar `count`, `positive count`, `positive rate`, `censoring` y `max spread`, sin incorporar tenure como feature del modelo.

3.2

Por lo tanto, para el EDA podemos analizar **observed customer history**, pero no llamarlo “true tenure”.

La definición correcta sería:

$$ObservedTenureMonths = MonthsBetween(ObservationDate, FirstObservedDataDate)$$

donde:

$$FirstObservedDataDate = \min(data_date)$$

por `party_id`.

El riesgo es **left censoring**: si muchos clientes aparecen por primera vez cerca del inicio histórico de `person_restricted`, ese `MIN(data_date)` no representa realmente cuándo comenzó la relación con el cliente.

Primero perfilaremos la fuente

Ejecuta:

```
eda_person_restricted_profile <- sdf_sql(  
  sc,  
  "  
  
  SELECT  
  
    COUNT(*) AS total_rows,  
  
    COUNT(DISTINCT party_id) AS distinct_parties,
```

```
MIN(data_date) AS min_data_date,  
MAX(data_date) AS max_data_date,  
SUM(CASE WHEN party_id IS NULL THEN 1 ELSE 0 END) AS null_party_id,  
SUM(CASE WHEN data_date IS NULL THEN 1 ELSE 0 END) AS null_data_date
```

```
FROM bc3.person_restricted
```

```
"
```

```
) %>%
```

```
collect()
```

```
eda_person_restricted_profile
```

Después construimos el primer registro observado por party y lo cruzamos contra nuestra población analítica:

```
eda_observed_tenure_profile <- sdf_sql(
```

```
sc,
```

```
"
```

```
WITH first_seen AS (
```

```
SELECT
```

```
party_id,
```

```
MIN(data_date) AS first_observed_date
```

```
FROM bc3.person_restricted
```

```
WHERE party_id IS NOT NULL
```

```
AND data_date IS NOT NULL
```

```
GROUP BY party_id  
)
```

```
cohort AS (
```

```
SELECT  
    party_id,  
    observation_date,  
    label_3m
```

```
FROM bu_frd.model_features_ab_final_2022_2025  
)
```

```
joined AS (
```

```
SELECT  
    c.party_id,  
    c.observation_date,  
    c.label_3m,  
    f.first_observed_date,
```

```
CASE  
    WHEN f.first_observed_date IS NOT NULL  
    THEN FLOOR(  
        MONTHS_BETWEEN(  
            c.observation_date,
```

```

        f.first_observed_date
    )
)
ELSE NULL
END AS observed_tenure_months

FROM cohort c

LEFT JOIN first_seen f
    ON c.party_id = f.party_id
)

SELECT
    COUNT(*) AS total_parties,

    SUM(
        CASE
            WHEN first_observed_date IS NULL THEN 1
            ELSE 0
        END
    ) AS missing_first_observed_date,

    MIN(first_observed_date) AS min_first_observed_date,

    MAX(first_observed_date) AS max_first_observed_date,

```

AVG(observed_tenure_months) AS avg_observed_tenure_months,

MIN(observed_tenure_months) AS min_observed_tenure_months,

percentile_approx(observed_tenure_months, 0.01) AS p01,

percentile_approx(observed_tenure_months, 0.05) AS p05,

percentile_approx(observed_tenure_months, 0.25) AS p25,

percentile_approx(observed_tenure_months, 0.50) AS p50,

percentile_approx(observed_tenure_months, 0.75) AS p75,

percentile_approx(observed_tenure_months, 0.95) AS p95,

percentile_approx(observed_tenure_months, 0.99) AS p99,

MAX(observed_tenure_months) AS max_observed_tenure_months

FROM joined

"

) %>%

collect()

eda_observed_tenure_profile

También necesitamos saber si existe acumulación artificial en la primera fecha disponible de la fuente. Este query es especialmente importante:

eda_first_seen_distribution <- sdf_sql(

sc,

"

WITH first_seen AS (

```

SELECT
    party_id,
    MIN(data_date) AS first_observed_date

FROM bc3.person_restricted

WHERE party_id IS NOT NULL
    AND data_date IS NOT NULL

GROUP BY party_id
),

cohort AS (

    SELECT DISTINCT party_id
    FROM bu_frd.model_features_ab_final_2022_2025
)

SELECT
    YEAR(f.first_observed_date) AS first_seen_year,
    COUNT(*) AS parties

FROM cohort c

INNER JOIN first_seen f
    ON c.party_id = f.party_id

```

```
GROUP BY YEAR(f.first_observed_date)
```

```
ORDER BY first_seen_year
```

```
"
```

```
) %>%
```

```
collect()
```

eda_first_seen_distribution

Con esos tres resultados decidiremos cómo hacer los buckets.

La decisión metodológica aquí será importante: si observamos una gran concentración en el primer año disponible, **no eliminaremos esta sección del EDA**, pero la presentaremos como:

Observed-history confound analysis

en vez de:

Customer tenure analysis.

Y probablemente separaremos:

Potentially left-censored

vs

more reliably observed

antes de calcular positive rates por duración observada.

Pásame los resultados de esos tres queries y definimos los buckets sin asumir arbitrariamente dónde empieza el censoring.

3.3

```
eda_source_start_censoring <- sdf_sql(  
  sc,  
  "  
  WITH cohort AS (  
    SELECT  
      party_id,  
      label_3m  
    FROM bu_frd.model_features_ab_final_2022_2025  
  ),  
  
  source_start AS (  
    SELECT DISTINCT  
      party_id  
    FROM bc3.person_restricted  
    WHERE data_date = DATE '2000-09-30'  
  )  
  
  SELECT  
    COUNT(*) AS total_parties,  
  
    SUM(  
      CASE WHEN s.party_id IS NOT NULL  
        THEN 1 ELSE 0 END  
    ) AS source_start_parties,  
  
    AVG(  
      CASE WHEN s.party_id IS NOT NULL  
        THEN 1.0 ELSE 0.0 END  
    ) AS source_start_pct,  
  
    SUM(  
      CASE WHEN s.party_id IS NOT NULL  
        AND c.label_3m = 1  
        THEN 1 ELSE 0 END  
    ) AS source_start_positive,  
  
    AVG(  
      CASE WHEN s.party_id IS NOT NULL  
        THEN c.label_3m * 1.0  
        ELSE NULL END  
    ) AS source_start_positive_rate,  
  
    AVG(  
      CASE WHEN s.party_id IS NULL  
        THEN c.label_3m * 1.0  
        ELSE NULL END
```

```

    ) AS later_history_positive_rate

FROM cohort c

LEFT JOIN source_start s
  ON c.party_id = s.party_id
"
) %>%
  collect()

eda_source_start_censoring

```

4- EDA: **Credit Exposure & Portfolio Structure.**

1. Exposición crediticia por party

Vamos a resumir los 12 meses disponibles para cada party y después comparar únicamente `label = 0` vs `label = 1`.

```

eda_credit_exposure <- sdf_sql(
  sc,
  "
  WITH party_exposure AS (

    SELECT
      party_id,

      AVG(reported_accounts) AS avg_reported_accounts,
      AVG(active_accounts) AS avg_active_accounts,

      SUM(
        CASE WHEN active_accounts = 0
          THEN 1 ELSE 0 END
      ) AS zero_active_months,

      COUNT(*) AS observed_months

    FROM bu_frd.model_party_behavior_monthly_2022_2025

    GROUP BY party_id
  ),

  base AS (

    SELECT
      c.party_id,
      c.label_3m,

```

```

        p.avg_reported_accounts,
        p.avg_active_accounts,
        p.zero_active_months,
        p.observed_months

FROM bu_frd.model_features_ab_final_2022_2025 c

LEFT JOIN party_exposure p
  ON c.party_id = p.party_id
)

SELECT
  label_3m,

  COUNT(*) AS parties,

  AVG(avg_reported_accounts) AS avg_reported_accounts,
  percentile_approx(avg_reported_accounts, 0.50)
  AS median_reported_accounts,

  AVG(avg_active_accounts) AS avg_active_accounts,
  percentile_approx(avg_active_accounts, 0.50)
  AS median_active_accounts,

  AVG(
    CASE
      WHEN zero_active_months = observed_months
      THEN 1.0 ELSE 0.0
    END
  ) AS pct_parties_zero_active_all_months,

  AVG(
    zero_active_months * 1.0 / observed_months
  ) AS avg_pct_months_zero_active

FROM base

GROUP BY label_3m

ORDER BY label_3m
"
) %>%
collect()

eda_credit_exposure

```

Esto nos responderá algo estructural:

¿Los positive y negative observations tienen niveles diferentes de credit-file exposure?

No lo vamos a interpretar todavía como “feature strength”; solo como una posible diferencia de composición que debemos conocer antes del modelado.

2. Distribución de active-account exposure

El promedio por sí solo no es suficiente. Hagamos buckets sencillos utilizando el promedio mensual de cuentas activas por party:

```
eda_active_account_buckets <- sdf_sql(
  sc,
  "
  WITH party_exposure AS (

    SELECT
      party_id,
      AVG(active_accounts) AS avg_active_accounts

    FROM bu_frd.model_party_behavior_monthly_2022_2025

    GROUP BY party_id
  ),

  base AS (

    SELECT
      c.party_id,
      c.label_3m,
      p.avg_active_accounts,

    CASE
      WHEN p.avg_active_accounts = 0
        THEN '0'
      WHEN p.avg_active_accounts <= 1
        THEN '>0 to 1'
      WHEN p.avg_active_accounts <= 3
        THEN '>1 to 3'
      WHEN p.avg_active_accounts <= 5
        THEN '>3 to 5'
      WHEN p.avg_active_accounts <= 10
        THEN '>5 to 10'
      ELSE '>10'
    END AS active_account_bucket
```

```

FROM bu_frd.model_features_ab_final_2022_2025 c

LEFT JOIN party_exposure p
  ON c.party_id = p.party_id
)

SELECT
  active_account_bucket,

  COUNT(*) AS parties,

  SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END)
  AS positive_cases,

  AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END)
  AS positive_rate

FROM base

GROUP BY active_account_bucket

ORDER BY
  CASE active_account_bucket
    WHEN '0' THEN 1
    WHEN '>0 to 1' THEN 2
    WHEN '>1 to 3' THEN 3
    WHEN '>3 to 5' THEN 4
    WHEN '>5 to 10' THEN 5
    WHEN '>10' THEN 6
  END
"
) %>%
collect()

```

eda_active_account_buckets

Esta será una buena candidata para gráfica:

Average active-account exposure

↓
 0 | >0-1 | >1-3 | >3-5 | >5-10 | >10

y-axis:

party count

o positive rate

3. Antes de hacer Product / Portfolio Analysis

Aquí no quiero asumir nombres de columnas. Revisemos únicamente metadata, lo cual debería ser prácticamente inmediato y no requiere scan de datos.

Ejecuta:

```
monthly_cols <- colnames(  
  tbl(sc, "bu_frd.model_party_behavior_monthly_2022_2025")  
)
```

```
monthly_cols[  
  grepl(  
    "account|acct|portfolio|product|util",  
    monthly_cols,  
    ignore.case = TRUE  
  )  
]
```

Y también:

```
trade_cols <- colnames(  
  tbl(sc, "bu_frd.model_trades_behavioral_final_13m_2022_2025")  
)
```

```
trade_cols[  
  grepl(  
    "portfolio|product|type|acct|account",  
    trade_cols,  
    ignore.case = TRUE  
  )  
]
```

Esto nos dirá si podemos construir de forma confiable una sección de:

Portfolio / Product Mix

sin regresar al dataset raw de cientos de miles de millones de filas.

Qué espero cerrar con esta sección

Con estos resultados decidiremos si el EDA tendrá:

Credit Exposure — sí, con seguridad.

Y:

Product/Portfolio composition — sí o no, dependiendo de qué dimensión validada exista realmente en la extracción de trades.

No necesitamos volver a `bc3.person_restricted`.

Pásame los resultados de los **dos queries de exposure** y las columnas que aparezcan en los dos `colnames()`. Con eso construimos la siguiente parte sin introducir análisis que pertenezca al Feature Validation Report.

4 -

Perfecto. Con esas columnas podemos aislar correctamente los **12 meses analíticos** usando:

`month_offset BETWEEN 0 AND 11`

y excluir automáticamente:

`month_offset = 12 → SEED`

Para esta sección del EDA recomiendo hacer el análisis a nivel **party-product exposure**, no a nivel de filas mensuales. Si contáramos directamente las filas de trades, una cuenta reportada durante 12 meses podría aparecer 12 veces y distorsionar el product mix.

1. Primero perfilamos `acct_typ_cde`

```
eda_account_type_profile <- sdf_sql(  
  sc,  
  "  
  WITH analytic_trades AS (  
  
    SELECT  
      party_id,  
      acct_id,  
      acct_typ_cde  
  
    FROM bu_frd.model_trades_behavioral_final_13m_2022_2025  
  
    WHERE month_offset BETWEEN 0 AND 11  
      AND acct_typ_cde IS NOT NULL  
  
  ),  
  
  party_account_type AS (  
  
    SELECT DISTINCT  
      party_id,  
      acct_id,  
      acct_typ_cde  
  
    FROM analytic_trades  
  )  
  
  SELECT
```

```

    acct_typ_cde,

    COUNT(DISTINCT acct_id) AS distinct_accounts,

    COUNT(DISTINCT party_id) AS distinct_parties,

    COUNT(*) AS party_account_combinations

FROM party_account_type

GROUP BY acct_typ_cde

ORDER BY distinct_parties DESC
"
) %>%
collect()

```

eda_account_type_profile

Esto nos mostrará qué códigos existen realmente y cuáles concentran el volumen.

No les pondremos nombres de producto todavía hasta confirmar el mapping oficial de [acct_typ_cde](#).

2. Product/account-type exposure vs outcome

Después calculamos exposición por tipo:

```

eda_account_type_outcome <- sdf_sql(
  sc,
  "
  WITH party_type AS (

    SELECT DISTINCT
      party_id,
      acct_typ_cde

    FROM bu_frd.model_trades_behavioral_final_13m_2022_2025

    WHERE month_offset BETWEEN 0 AND 11
      AND acct_typ_cde IS NOT NULL

  ),

  base AS (

```

```

SELECT
    p.party_id,
    p.acct_typ_cde,
    c.label_3m

FROM party_type p

INNER JOIN bu_frd.model_features_ab_final_2022_2025 c
    ON p.party_id = c.party_id
)

SELECT
    acct_typ_cde,

    COUNT(*) AS parties_with_type,

    SUM(
        CASE WHEN label_3m = 1
            THEN 1 ELSE 0 END
    ) AS positive_parties,

    SUM(
        CASE WHEN label_3m = 0
            THEN 1 ELSE 0 END
    ) AS negative_parties,

    AVG(
        CASE WHEN label_3m = 1
            THEN 1.0 ELSE 0.0 END
    ) AS positive_rate

FROM base

GROUP BY acct_typ_cde

ORDER BY parties_with_type DESC
"
) %>%
collect()

```

eda_account_type_outcome

Aquí hay una consideración importante: **las categorías no serán mutuamente excluyentes**.

Una persona puede tener:

acct type A

+
acct type B
+
acct type C

y aparecerá una vez en cada una.

Eso es correcto porque la pregunta es:

“¿Qué proporción de parties expuestos a este account type pertenece al grupo positivo?”

No estamos tratando de dividir la población en productos exclusivos.

3. Product breadth

También recomiendo medir cuántos tipos distintos de cuenta tiene cada party. Esto complementa muy bien el análisis de active accounts porque distingue entre:

muchas cuentas del mismo tipo

y

diversidad de productos

```
eda_product_breadth <- sdf_sql(  
  sc,  
  "
```

```
  WITH party_product_count AS (  
    SELECT
```

```
      party_id,
```

```
      COUNT(DISTINCT acct_typ_cde) AS distinct_account_types
```

```
    FROM bu_frd.model_trades_behavioral_final_13m_2022_2025
```

```
    WHERE month_offset BETWEEN 0 AND 11
```

```
      AND acct_typ_cde IS NOT NULL
```

```
    GROUP BY party_id
```

```
  ),  
  base AS (  
    SELECT
```

```
      c.party_id,
```

```
      c.label_3m,
```

```
      COALESCE(p.distinct_account_types,0)
```

```

        AS distinct_account_types

FROM bu_frd.model_features_ab_final_2022_2025 c

LEFT JOIN party_product_count p
    ON c.party_id = p.party_id
),

bucketed AS (

SELECT
    *,

    CASE
        WHEN distinct_account_types = 0 THEN '0'
        WHEN distinct_account_types = 1 THEN '1'
        WHEN distinct_account_types = 2 THEN '2'
        WHEN distinct_account_types = 3 THEN '3'
        WHEN distinct_account_types = 4 THEN '4'
        ELSE '5+'
    END AS product_breadth_bucket

FROM base
)

SELECT
    product_breadth_bucket,

    COUNT(*) AS parties,

    SUM(
        CASE WHEN label_3m = 1
            THEN 1 ELSE 0 END
    ) AS positive_cases,

    AVG(
        CASE WHEN label_3m = 1
            THEN 1.0 ELSE 0.0 END
    ) AS positive_rate

FROM bucketed

GROUP BY product_breadth_bucket

ORDER BY
    CASE product_breadth_bucket
        WHEN '0' THEN 1
        WHEN '1' THEN 2

```

```

        WHEN '2' THEN 3
        WHEN '3' THEN 4
        WHEN '4' THEN 5
        WHEN '5+' THEN 6
    END
"
) %>%
  collect()

eda_product_breadth

```

4- trayectoria mensual

```

eda_behavior_monthly <- sdf_sql(
  sc,
  "
SELECT
  month_offset,

  CASE
    WHEN month_offset = 0 THEN 'M0'
    ELSE CONCAT('M-', CAST(month_offset AS STRING))
  END AS relative_month,

  feature_window,
  label_3m,

  COUNT(*) AS party_months,

  AVG(reported_accounts) AS avg_reported_accounts,
  AVG(active_accounts) AS avg_active_accounts,

  AVG(deterioration_count) AS avg_deterioration_count,
  AVG(improvement_count) AS avg_improvement_count,

  AVG(adverse_entry_count) AS avg_adverse_entry_count,
  AVG(adverse_resolved_count) AS avg_adverse_resolved_count,

  AVG(max_dpd_jump) AS avg_max_dpd_jump,

  AVG(avg_balance_active) AS avg_balance_active,

  AVG(avg_past_due_active) AS avg_past_due_active,

  AVG(delinquent_account_share) AS avg_delinquent_share,

```

```

    AVG(avg_utilization) AS avg_utilization,

    AVG(utilization_eligible_accounts)
    AS avg_utilization_eligible_accounts,

    AVG(rated_active_accounts)
    AS avg_rated_active_accounts,

    AVG(delinquent_active_accounts)
    AS avg_delinquent_active_accounts

FROM bu_frd.model_party_behavior_monthly_2022_2025

WHERE month_offset BETWEEN 0 AND 11

GROUP BY
    month_offset,
    feature_window,
    label_3m

ORDER BY
    month_offset DESC,
    label_3m
"
) %>%
collect()

eda_behavior_monthly

```

Un QA pequeño del resultado

Después del query corre:

```

eda_behavior_monthly %>%
select(
  month_offset,
  relative_month,
  feature_window,
  label_3m,
  party_months
)

```

5- Missingness & Availability Analysis

```
eda_missingness_profile <- sdf_sql(
```

SC,
"

WITH agg AS (

SELECT

COUNT(*) AS total_rows,

SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END)

AS negative_rows,

SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END)

AS positive_rows,

/* reported accounts */

SUM(CASE WHEN reported_accounts IS NULL THEN 1 ELSE 0 END)

AS reported_null,

AVG(

CASE WHEN label_3m = 0

THEN CASE WHEN reported_accounts IS NOT NULL THEN 1.0 ELSE 0.0 END

END

) AS reported_avail_neg,

AVG(

CASE WHEN label_3m = 1

THEN CASE WHEN reported_accounts IS NOT NULL THEN 1.0 ELSE 0.0 END

END

) AS reported_avail_pos,

/* active accounts */

SUM(CASE WHEN active_accounts IS NULL THEN 1 ELSE 0 END)

AS active_null,

AVG(

CASE WHEN label_3m = 0

THEN CASE WHEN active_accounts IS NOT NULL THEN 1.0 ELSE 0.0 END

END

) AS active_avail_neg,

AVG(

CASE WHEN label_3m = 1

THEN CASE WHEN active_accounts IS NOT NULL THEN 1.0 ELSE 0.0 END

END

) AS active_avail_pos,

/* balance */

SUM(CASE WHEN avg_balance_active IS NULL THEN 1 ELSE 0 END)

AS balance_null,

```
        AVG(
            CASE WHEN label_3m = 0
                THEN CASE WHEN avg_balance_active IS NOT NULL THEN 1.0 ELSE 0.0
            END
        ) AS balance_avail_neg,
```

```
        AVG(
            CASE WHEN label_3m = 1
                THEN CASE WHEN avg_balance_active IS NOT NULL THEN 1.0 ELSE 0.0
            END
        ) AS balance_avail_pos,
```

```
/* past due */
SUM(CASE WHEN avg_past_due_active IS NULL THEN 1 ELSE 0 END)
    AS pastdue_null,
```

```
        AVG(
            CASE WHEN label_3m = 0
                THEN CASE WHEN avg_past_due_active IS NOT NULL THEN 1.0 ELSE 0.0
            END
        ) AS pastdue_avail_neg,
```

```
        AVG(
            CASE WHEN label_3m = 1
                THEN CASE WHEN avg_past_due_active IS NOT NULL THEN 1.0 ELSE 0.0
            END
        ) AS pastdue_avail_pos,
```

```
/* delinquency share */
SUM(CASE WHEN delinquent_account_share IS NULL THEN 1 ELSE 0 END)
    AS delinquency_null,
```

```
        AVG(
            CASE WHEN label_3m = 0
                THEN CASE WHEN delinquent_account_share IS NOT NULL THEN 1.0 ELSE
0.0 END
            END
        ) AS delinquency_avail_neg,
```

```
        AVG(
            CASE WHEN label_3m = 1
                THEN CASE WHEN delinquent_account_share IS NOT NULL THEN 1.0 ELSE
0.0 END
```

```

        END
    ) AS delinquency_avail_pos,

    /* utilization */
    SUM(CASE WHEN avg_utilization IS NULL THEN 1 ELSE 0 END)
        AS utilization_null,

    AVG(
        CASE WHEN label_3m = 0
            THEN CASE WHEN avg_utilization IS NOT NULL THEN 1.0 ELSE 0.0 END
        END
    ) AS utilization_avail_neg,

    AVG(
        CASE WHEN label_3m = 1
            THEN CASE WHEN avg_utilization IS NOT NULL THEN 1.0 ELSE 0.0 END
        END
    ) AS utilization_avail_pos,

    /* utilization eligible accounts */
    SUM(
        CASE WHEN utilization_eligible_accounts IS NULL
            THEN 1 ELSE 0 END
    ) AS util_eligible_null,

    AVG(
        CASE WHEN label_3m = 0
            THEN CASE WHEN utilization_eligible_accounts IS NOT NULL
                THEN 1.0 ELSE 0.0 END
        END
    ) AS util_eligible_avail_neg,

    AVG(
        CASE WHEN label_3m = 1
            THEN CASE WHEN utilization_eligible_accounts IS NOT NULL
                THEN 1.0 ELSE 0.0 END
        END
    ) AS util_eligible_avail_pos,

    /* rated accounts */
    SUM(CASE WHEN rated_active_accounts IS NULL THEN 1 ELSE 0 END)
        AS rated_null,

    AVG(
        CASE WHEN label_3m = 0
            THEN CASE WHEN rated_active_accounts IS NOT NULL THEN 1.0 ELSE 0.0
END
        END

```

```

    ) AS rated_avail_neg,

    AVG(
      CASE WHEN label_3m = 1
        THEN CASE WHEN rated_active_accounts IS NOT NULL THEN 1.0 ELSE 0.0
      END
    ) AS rated_avail_pos

FROM bu_frd.model_party_behavior_monthly_2022_2025

WHERE month_offset BETWEEN 0 AND 11
)

SELECT
  metric,
  null_count,
  null_count * 1.0 / total_rows AS null_pct,
  availability_negative,
  availability_positive

FROM agg

LATERAL VIEW STACK(
  8,

  'reported_accounts',
  reported_null,
  reported_avail_neg,
  reported_avail_pos,

  'active_accounts',
  active_null,
  active_avail_neg,
  active_avail_pos,

  'avg_balance_active',
  balance_null,
  balance_avail_neg,
  balance_avail_pos,

  'avg_past_due_active',
  pastdue_null,
  pastdue_avail_neg,
  pastdue_avail_pos,

  'delinquent_account_share',
  delinquency_null,

```

```

delinquency_avail_neg,
delinquency_avail_pos,

'avg_utilization',
utilization_null,
utilization_avail_neg,
utilization_avail_pos,

'utilization_eligible_accounts',
util_eligible_null,
util_eligible_avail_neg,
util_eligible_avail_pos,

'rated_active_accounts',
rated_null,
rated_avail_neg,
rated_avail_pos

) s AS metric,
    null_count,
    availability_negative,
    availability_positive

ORDER BY null_pct DESC
"
) %>%
collect()

```

eda_missingness_profile

6- Data Quality & Temporal Integrity

```

eda_monthly_integrity <- sdf_sql(
  sc,
  "
WITH party_qa AS (

SELECT
  party_id,

  COUNT(*) AS row_count,

  COUNT(DISTINCT month_offset)
  AS distinct_month_offsets,

  MIN(month_offset) AS min_offset,
  MAX(month_offset) AS max_offset,

```

```

SUM(
  CASE
    WHEN feature_month > observation_date
    THEN 1 ELSE 0
  END
) AS future_feature_months,

SUM(
  CASE
    WHEN feature_month !=
      LAST_DAY(
        ADD_MONTHS(
          observation_date,
          -month_offset
        )
      )
    THEN 1 ELSE 0
  END
) AS misaligned_feature_months,

SUM(
  CASE
    WHEN month_offset BETWEEN 0 AND 2
      AND feature_window != 'RECENT_3M'
    THEN 1

    WHEN month_offset BETWEEN 3 AND 11
      AND feature_window != 'PRIOR_9M'
    THEN 1

    ELSE 0
  END
) AS invalid_window_mapping

FROM bu_frd.model_party_behavior_monthly_2022_2025

WHERE month_offset BETWEEN 0 AND 11

GROUP BY party_id
)

SELECT

COUNT(*) AS total_parties,

SUM(row_count) AS total_party_month_rows,

```

```

SUM(
  CASE WHEN row_count != 12
    THEN 1 ELSE 0 END
) AS parties_not_12_months,

SUM(
  CASE WHEN distinct_month_offsets != 12
    THEN 1 ELSE 0 END
) AS parties_duplicate_or_missing_offsets,

SUM(future_feature_months)
  AS future_feature_months,

SUM(misaligned_feature_months)
  AS misaligned_feature_months,

SUM(invalid_window_mapping)
  AS invalid_window_mapping,

MIN(min_offset) AS min_month_offset,

MAX(max_offset) AS max_month_offset

FROM party_qa
"
) %>%
collect()

eda_monthly_integrity

```

7-Raw Distribution & Outlier Review.

```

eda_raw_distributions <- sdf_sql(
  sc,
  "
SELECT

  /* Reported accounts */
  MIN(reported_accounts) AS reported_min,
  percentile_approx(reported_accounts, 0.01) AS reported_p01,
  percentile_approx(reported_accounts, 0.05) AS reported_p05,
  percentile_approx(reported_accounts, 0.25) AS reported_p25,
  percentile_approx(reported_accounts, 0.50) AS reported_p50,
  percentile_approx(reported_accounts, 0.75) AS reported_p75,
  percentile_approx(reported_accounts, 0.95) AS reported_p95,
  percentile_approx(reported_accounts, 0.99) AS reported_p99,
  MAX(reported_accounts) AS reported_max,

```

/* Active accounts */

MIN(active_accounts) AS active_min,
percentile_approx(active_accounts, 0.01) AS active_p01,
percentile_approx(active_accounts, 0.05) AS active_p05,
percentile_approx(active_accounts, 0.25) AS active_p25,
percentile_approx(active_accounts, 0.50) AS active_p50,
percentile_approx(active_accounts, 0.75) AS active_p75,
percentile_approx(active_accounts, 0.95) AS active_p95,
percentile_approx(active_accounts, 0.99) AS active_p99,
MAX(active_accounts) AS active_max,

/* Average balance */

MIN(avg_balance_active) AS balance_min,
percentile_approx(avg_balance_active, 0.01) AS balance_p01,
percentile_approx(avg_balance_active, 0.05) AS balance_p05,
percentile_approx(avg_balance_active, 0.25) AS balance_p25,
percentile_approx(avg_balance_active, 0.50) AS balance_p50,
percentile_approx(avg_balance_active, 0.75) AS balance_p75,
percentile_approx(avg_balance_active, 0.95) AS balance_p95,
percentile_approx(avg_balance_active, 0.99) AS balance_p99,
percentile_approx(avg_balance_active, 0.999) AS balance_p999,
MAX(avg_balance_active) AS balance_max,

/* Average past due */

MIN(avg_past_due_active) AS pastdue_min,
percentile_approx(avg_past_due_active, 0.50) AS pastdue_p50,
percentile_approx(avg_past_due_active, 0.75) AS pastdue_p75,
percentile_approx(avg_past_due_active, 0.95) AS pastdue_p95,
percentile_approx(avg_past_due_active, 0.99) AS pastdue_p99,
percentile_approx(avg_past_due_active, 0.999) AS pastdue_p999,
MAX(avg_past_due_active) AS pastdue_max,

/* Delinquency share */

MIN(delinquent_account_share) AS delinq_min,
percentile_approx(delinquent_account_share, 0.50) AS delinq_p50,
percentile_approx(delinquent_account_share, 0.75) AS delinq_p75,
percentile_approx(delinquent_account_share, 0.95) AS delinq_p95,
percentile_approx(delinquent_account_share, 0.99) AS delinq_p99,
MAX(delinquent_account_share) AS delinq_max,

/* Utilization */

MIN(avg_utilization) AS util_min,
percentile_approx(avg_utilization, 0.01) AS util_p01,
percentile_approx(avg_utilization, 0.05) AS util_p05,
percentile_approx(avg_utilization, 0.25) AS util_p25,
percentile_approx(avg_utilization, 0.50) AS util_p50,
percentile_approx(avg_utilization, 0.75) AS util_p75,
percentile_approx(avg_utilization, 0.95) AS util_p95,

```
percentile_approx(avg_utilization, 0.99) AS util_p99,
percentile_approx(avg_utilization, 0.999) AS util_p999,
MAX(avg_utilization) AS util_max,
```

```
/* Behavioral events */
```

```
percentile_approx(deterioration_count, 0.50) AS deterioration_p50,
percentile_approx(deterioration_count, 0.95) AS deterioration_p95,
percentile_approx(deterioration_count, 0.99) AS deterioration_p99,
MAX(deterioration_count) AS deterioration_max,
```

```
percentile_approx(adverse_entry_count, 0.50) AS adverse_p50,
percentile_approx(adverse_entry_count, 0.95) AS adverse_p95,
percentile_approx(adverse_entry_count, 0.99) AS adverse_p99,
MAX(adverse_entry_count) AS adverse_max,
```

```
percentile_approx(max_dpd_jump, 0.50) AS dpdjump_p50,
percentile_approx(max_dpd_jump, 0.95) AS dpdjump_p95,
percentile_approx(max_dpd_jump, 0.99) AS dpdjump_p99,
MAX(max_dpd_jump) AS dpdjump_max
```

```
FROM bu_frd.model_party_behavior_monthly_2022_2025
```

```
WHERE month_offset BETWEEN 0 AND 11
```

```
"
```

```
) %>%
```

```
collect()
```

```
eda_raw_distributions
```

QA separado:

```
eda_logical_qa <- sdf_sql(
```

```
sc,
```

```
"
```

```
SELECT
```

```
COUNT(*) AS total_rows,
```

```
SUM(CASE WHEN reported_accounts < 0 THEN 1 ELSE 0 END)
AS negative_reported_accounts,
```

```
SUM(CASE WHEN active_accounts < 0 THEN 1 ELSE 0 END)
AS negative_active_accounts,
```

```
SUM(CASE WHEN active_accounts > reported_accounts THEN 1 ELSE 0 END)
AS active_gt_reported,
```

```

SUM(CASE WHEN rated_active_accounts > active_accounts THEN 1 ELSE 0 END)
    AS rated_gt_active,

SUM(CASE WHEN delinquent_active_accounts > rated_active_accounts THEN 1 ELSE
0 END)
    AS delinquent_gt_rated,

SUM(CASE WHEN utilization_eligible_accounts > active_accounts THEN 1 ELSE 0
END)
    AS util_eligible_gt_active,

SUM(
    CASE
        WHEN delinquent_account_share < 0
            OR delinquent_account_share > 1
        THEN 1 ELSE 0
    END
) AS invalid_delinquent_share,

SUM(CASE WHEN deterioration_count < 0 THEN 1 ELSE 0 END)
    AS negative_deterioration,

SUM(CASE WHEN improvement_count < 0 THEN 1 ELSE 0 END)
    AS negative_improvement,

SUM(CASE WHEN adverse_entry_count < 0 THEN 1 ELSE 0 END)
    AS negative_adverse_entries,

SUM(CASE WHEN max_dpd_jump < 0 THEN 1 ELSE 0 END)
    AS negative_max_dpd_jump

FROM bu_frd.model_party_behavior_monthly_2022_2025

WHERE month_offset BETWEEN 0 AND 11
"
) %>%
collect()

eda_logical_qa

```